EXPERIMENT 5

AIM : To implement a Support Vector Machine (SVM) classifier for predicting diabetes and improve its performance using hyperparameter tuning techniques.

THEORY

1. Dataset Source

Dataset Name: Pima Indians Diabetes Dataset

Dataset Link:
https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

Original Repository: UCI Machine Learning Repository

2. Dataset Overview

The **Pima Indians Diabetes dataset** is widely used in machine learning for **binary classification problems in healthcare analytics**. The aim of this dataset is to determine whether a patient suffers from diabetes based on several medical measurements collected during clinical examinations.

Dataset Properties

- **Total number of observations:** 768
- **Number of input attributes:** 8
- **Target variable:** Outcome
- **Learning type:** Supervised learning (Binary classification)

The target variable represents whether a patient is diabetic or not and can be expressed as:

Outcome={0,if the patient is non-diabetic1,if the patient is diabeticOutcome={0,1,if the patient is non-diabeticif the patient is diabetic

Feature Description

The dataset contains the following medical attributes:

1. **Pregnancies** – Total number of pregnancies experienced by the patient
2. **Glucose** – Plasma glucose concentration measured during an oral glucose tolerance test
3. **BloodPressure** – Diastolic blood pressure value (mm Hg)
4. **SkinThickness** – Thickness of triceps skin fold (mm)
5. **Insulin** – Serum insulin concentration after 2 hours (mu U/ml)
6. **BMI** – Body Mass Index calculated using

7. **DiabetesPedigreeFunction** – Score representing genetic predisposition to diabetes
8. **Age** – Age of the patient (years)

Target Variable

The classification output is represented by:

- **0 → Non-diabetic patient**
- **1 → Diabetic patient**

3. Mathematical Formulation of Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for **classification and regression tasks**. It works by constructing an optimal decision boundary that maximizes the separation between different classes.

Decision Boundary

For a linearly separable dataset, SVM identifies a **hyperplane** defined as

Where:

- ww = weight vector
- xx = feature vector
- bb = bias term

The classification rule can be expressed as:

- $f(x)$=+1f(x)=+1 indicates the positive class
- $f(x)$=−1f(x)=−1 indicates the negative class

Margin Maximization

SVM attempts to maximize the **margin**, which is the distance between the hyperplane and the nearest data points (called support vectors).

To achieve the optimal margin, SVM solves the following optimization problem

Kernel Trick

When the data is **not linearly separable**, SVM applies a **kernel function** to transform the data into a higher dimensional space.

One commonly used kernel is the **Radial Basis Function (RBF)**

Where:

- γγ determines how far the influence of a training point reaches.

4. Limitations of SVM

Despite its effectiveness, SVM has several practical limitations:

1. It is highly sensitive to **feature scaling**, therefore normalization is necessary.
2. Training time increases significantly for **very large datasets**.
3. Selecting the appropriate **kernel function** can be challenging.
4. Performance decreases when **classes overlap significantly**.
5. Standard SVM does not directly provide **probability outputs**.
6. It is susceptible to **noise and outliers** in the dataset.

SVM may perform poorly when:

- The dataset contains a high amount of noise.
- The number of samples is extremely large.
- Features are not properly normalized.

5. Methodology / Workflow

The following procedure was used to implement the model.

Step 1: Data Loading

The dataset **diabetes.csv** was uploaded into the environment and imported using the **pandas library**.

Step 2: Data Preprocessing

- The dataset structure was inspected.
- Input variables (**X**) and output variable (**y**) were separated.
- Feature normalization was performed using **StandardScaler**

Where:

- xx = original feature value
- μμ = mean of feature
- σσ = standard deviation

Step 3: Train-Test Splitting

The dataset was divided into two subsets:

- **Training Data:** 80%
- **Testing Data:** 20%

This ensures that the model learns from one portion of the data and is evaluated on unseen data.

Step 4: Model Implementation

A **Support Vector Classifier (SVC)** was implemented using the **scikit-learn library**.

Step 5: Hyperparameter Optimization

To improve model performance, **GridSearchCV** was used with **5-fold cross-validation**.

The following parameters were tuned:

- **C** – Regularization parameter controlling margin width
- **gamma** – Kernel coefficient controlling influence of data points
- **kernel** – Function used to map data into higher dimensions

Step 6: Model Evaluation

The model performance was assessed using:

- **Accuracy Score**
- **Classification Report** (Precision, Recall, F1-score)
- **Confusion Matrix**

6. Performance Analysis

Optimal Hyperparameters

After performing hyperparameter tuning, the following configuration produced the best results:

- **C = 100**
- **gamma = 0.001**
- **kernel = RBF**

Model Accuracy

Accuracy=77.27%

Confusion Matrix

	Predicted 0	**Predicted 1**
Actual 0	82	17
Actual 1	18	37

Result Interpretation

- **82 non-diabetic patients** were correctly identified.
- **37 diabetic patients** were correctly classified.
- **18 diabetic cases** were incorrectly predicted as healthy.
- **17 healthy patients** were incorrectly classified as diabetic.

This indicates that the classifier performs slightly better at identifying **non-diabetic cases**.

Weighted F1 Score

$F1=0.77$

7. Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** combined with **5-fold cross-validation** to ensure robust model evaluation.

Parameter Search Space

$C=[0.1,1,10,100]$
C=[0.1,1,10,100]
$\gamma=[1,0.1,0.01,0.001]$
γ=[1,0.1,0.01,0.001]
$Kernel=[RBF,Linear]$Kernel=[RBF,Linear]

Total Model Combinations

$4\times4\times2=324\times4\times2=32$

With **5-fold cross-validation**, the total number of training runs becomes:

$32\times5=16032\times5=160$

Best Configuration

$C=100,\gamma=0.001,Kernel=RBF$C=100,γ=0.001,Kernel=RBF

Impact of Hyperparameter Tuning

- Improved model **generalization ability**
- Reduced risk of **overfitting**
- Enhanced overall **classification performance**

Code:

```
# ========================================
# Import Libraries
# ========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# ==========================================
# Upload Dataset in Colab
# ==========================================
from google.colab import files
uploaded = files.upload()
# Load dataset
df = pd.read_csv('diabetes.csv')
# Display first 5 rows
print("First 5 rows of dataset:")
print(df.head())
# ==========================================
# Define Features and Target
# ==========================================
X = df.drop('Outcome', axis=1)
y = df['Outcome']
# ==========================================
# Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ==========================================
# Feature Scaling (VERY IMPORTANT for SVM)
# ==========================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# ==========================================
# Apply SVM with Hyperparameter Tuning
# ==========================================
# Define parameter grid
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}
# Create SVM model
svc = SVC()
# Grid Search
grid = GridSearchCV(
    svc,
    param_grid,
    refit=True,
    verbose=0,
    cv=5
)
# Train model
grid.fit(X_train, y_train)
# ==========================================
```

```
# Best Parameters
# ==========================================
print("\nBest Parameters Found:")
print(grid.best_params_)
# ==========================================
# Predictions
# ==========================================
y_pred = grid.predict(X_test)
# ==========================================
# Model Evaluation
# ==========================================
print("\nAccuracy Score:")
print(accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# ==========================================
# Confusion Matrix
# ==========================================
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

mldl5 yogita.ipynb

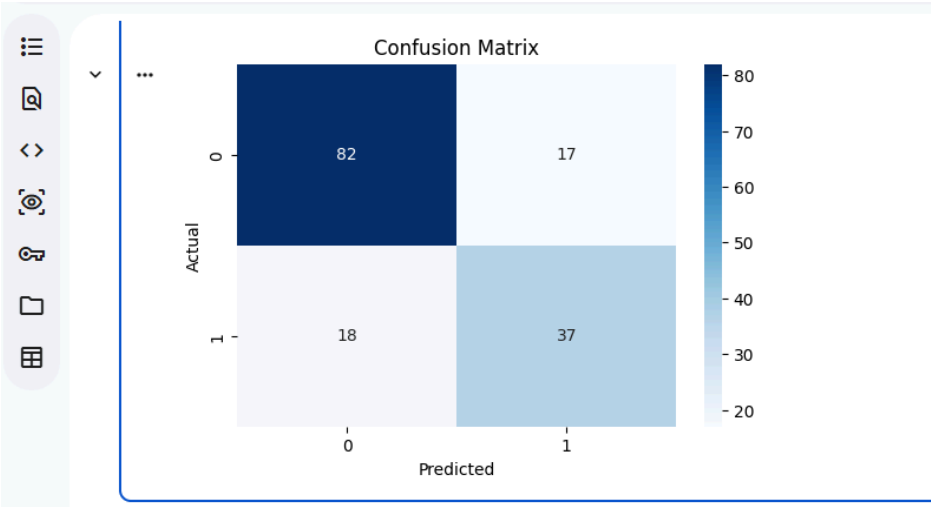
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Choose Files diabetes.csv

diabetes.csv(text/csv) - 23873 bytes, last modified: 3/16/2026 - 100% done

```
Saving diabetes.csv to diabetes (1).csv
First 5 rows of dataset:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Best Parameters Found:
{'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}

Accuracy Score:
0.7727272727272727

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.83      0.82        99
           1       0.69      0.67      0.68        55

    accuracy                           0.77       154
   macro avg       0.75      0.75      0.75       154
weighted avg       0.77      0.77      0.77       154
```



Conclusion

The **Support Vector Machine (SVM)** algorithm was applied to the **Pima Indians Diabetes dataset** for binary classification. Using **GridSearchCV** for hyperparameter tuning, the best model used the **RBF kernel** with **C = 100** and **gamma = 0.001**, achieving an accuracy of **77.27%**. The model showed better performance in predicting **non-diabetic cases** compared to diabetic cases.